

Lab 1 Report - 10/3/26

CSEN 383

Zachary Common and Calianna Collins

<https://github.com/Eval427/xv6-riscv>

https://youtu.be/uY5zhrfA_8M

Modified files

```
\ = Modified
+ = Added
- = Removed

\ Makefile

\ kernel/defs.h
\ kernel/kalloc.c
\ kernel/proc.c
\ kernel/proc.h
\ kernel/syscall.c
\ kernel/syscall.h
\ kernel/sysproc.c

+ user/lab1test.c
+ user/procinfo.c
+ user/sysinfo.c
\ user/user.h
\ user/usys.pl
+ user/test.c
```

Part 0

This part of the lab is the introduction on how to implement system calls into the xv6 OS. Since the instructions for this step are provided in the slides, we will show only our output

```
xv6 kernel is booting

init: starting sh
$ test
Say hello to kernel 0
Hello from the kernel space 0
$ test 10
```

```
Say hello to kernel 10
Hello from the kernel space 10
```

Part 1

The goal of part 1 was to provide the user with information about the system. Given an input of 0, 1, or 2, the total number of active processes, total system calls, or free memory pages in the system would be displayed. When the user inputs `sysinfo <0,1,2>`, the requested data is returned as an `int`.

Dataflow

1. Every system call is executed on the user side initially. Therefore, on the user side, it is first necessary to define the name of the system call and the function declaration to determine the number of user arguments that must be given to the call.

```
// user/user.h:25

int sysinfo(int); // sysinfo
```

and

```
// user/usys.pl:40

entry("sysinfo"); # sysinfo syscall for user
```

gets compiled into assembly in `usys.S` as

```
.global sysinfo
sysinfo:
li a7, SYS_sysinfo
ecall
ret
```

2. To provide optional functionality via the command line for debugging, a respective `main()` function is implemented in `sysinfo.c` and linked to the syscall. This is determined in `Makefile`:

```
# Makefile:99-143

# Links the call to its respective .o file by name
_%: %.o $(ULIB)

UPROGS=\
    $U/_cat\
    ...
```

```
$U/_test\
$U/_sysinfo\
...
```

and, for this part, implemented in `user/sysinfo.c`:

```
// user/sysinfo.c

#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int main(int argc, char *argv[]) {
    int n=0;
    if (argc >= 2) n = atoi(argv[1]);
    else {
        printf("Usage: sysinfo <0/1/2>\n"); // active processes/num
        syscalls/free mem pages
        return -1;
    }

    return sysinfo(n);
}
```

3. As soon as `sysinfo(n)` is called, the system call that was triggered and any parameters are stored in registers. The rest of the execution is then completed by the kernel (until the return value is passed back to the user on `user/sysinfo.c:13`). The table of entries in `syscall.h` allows the kernel and user space to reference the same syscall via a constant number.

```
// kernel/syscall.h:24

#define SYS_sysinfo 23 // sysinfo
```

These numbers are then mapped to functions in `syscall.c`:

```
// kernel/syscall.c:112-142
extern uint64 sys_sysinfo(void);

...

static uint64 (*syscalls[])(void) = {
    ...
    [SYS_sysinfo] sys_sysinfo, // sysinfo: syscall entry
    ...
};
```

And defined in `sysproc.c`:

```
// kernel/sysproc.c:104-113

// sysinfo syscall definition
uint64 sys_sysinfo(void) {
    int param;
    argint(0, &param);

    if (param >= 0 && param < 3) {
        return print_sysinfo(param);
    }

    return -1;
}
```

4. The function `print_sysinfo(int)` is implemented in `proc.c` not out of necessity, but rather ease of implementation. Most information required by this system call resides in `proc.c` and changes that were required to fetch additional information were also made in the same place. Once executed, the return value of the function is routed back to user space.

```
// kernel/proc.c:703-733

// sysinfo: print system info
int print_sysinfo(int n) {
    switch (n) {
        case 0:
            struct proc *p;
            int active_procs = 0;
            for (p = proc; p < &proc[NPROC]; p++) {
                acquire(&p->lock);
                if (p->state > 1) { // > 1 is not USED/UNUSED
                    active_procs++;
                }
                release(&p->lock);
            }
            //printf("%d active system processes\n", active_procs);
            return active_procs;

        case 1:
            int total_syscalls = get_total_syscalls();
            //printf("%d syscalls since boot\n", total_syscalls);
            return total_syscalls;

        case 2:
            int free_pages = freepages();
            //printf("%d available pages\n", free_pages);
            return free_pages;

        default:
```

```
    printf("Invalid input\n");  
    return -1;  
}  
}
```

Changes

1. In order to get the number of free pages in the system, a new function `freepages(void)` was implemented in `kernel/kalloc.c`. To fetch the total number of available pages, it starts from the first available page in the list of pages in kernel memory and iterates through the list of available pages until it reaches the end.

```
// kernel/kalloc.c:84-95  
  
// Fetch the number of free pages in the kernel  
int  
freepages(void)  
{  
    int count = 0;  
    struct run *r;  
  
    // kmem.freelist represents the first available page in the list  
    of pages in kernel memory  
    for (r = kmem.freelist; r != 0; r = r->next) count++;  
  
    return count;  
}
```

2. To track the total number of system calls executed since boot, we used an integer counter in `syscall.c` alongside a getter function:

```
// kernel/syscall.c:10-15  
  
// Record total number of system calls  
int total_syscalls = 0;  
  
int get_total_syscalls() {  
    return total_syscalls;  
}
```

and incremented the counter every time a system call was executed in `syscall(void)`:

```
// kernel.syscall.c:144-162  
  
void  
syscall(void)
```

```

{
    int num;
    struct proc *p = myproc();

    num = p->trapframe->a7;
    if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
        // Use num to lookup the system call function for num, call it,
        // and store its return value in p->trapframe->a0
        p->trapframe->a0 = syscalls[num]();
        total_syscalls++; // <---
    } else {
        printf("%d %s: unknown sys call %d\n",
            p->pid, p->name, num);
        p->trapframe->a0 = -1;
    }
}

```

Output (Assuming prints are active)

```

$ sysinfo 0
3 active system processes
$ sysinfo 1
128 syscalls since boot
$ sysinfo 2
32532 available pages

```

Part 2

Part 2 provides process-specific information when called. It requires the user to pass in an address to a `pinfo` struct as an argument and returns -1 or 0 on a fail/pass and fills the user's `pinfo` struct using `copyout()`. Since the dataflow was discussed in part 1, we will show and explain file changes one by one instead

Changes

1. First, while unnecessary, a `main()` function was made so that `procinfo` could be called via the command line interface

```

# Makefile:125-144

UPROGS=\
    ...
    $U/_procinfo\
    ...

```

```
// user/procinfo.c

#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

struct pinfo {
    int ppid;
    int syscall_count;
    int page_usage;
};

int main(int argc, char *argv[]) {
    uint64 pinfo_addr;

    if (argc < 2) {
        fprintf(2, "Usage: procinfo <pinfo pointer>\n");
        return -1;
    }

    pinfo_addr = (uint64)atoi(argv[1]);

    return procinfo((struct pinfo *)pinfo_addr);
}
```

2. Next, we defined the struct in kernel space so that it could be referenced

```
// kernel/proc.h

// Process info
struct pinfo {
    int ppid; // Parent process ID
    int syscall_count; // Total number of system calls that
the current process has made
    int page_usage; // Current process' memory size
};

// kernel/defs.h

struct pinfo;

// kalloc.c
int freepages(void);

// proc.c
int generate_procinfo(struct pinfo*); // procinfo

// syscall.c
int get_total_syscalls(void);
```

3. The `struct proc` also had to be adjusted to add a syscall counter to it to be referenced later

```
// kernel/proc.h

struct proc {
    struct spinlock lock;
    // p->lock must be held when using these:
    enum procstate state;           // Process state
    void *chan;                     // If non-zero, sleeping on chan
    int killed;                     // If non-zero, have been killed
    int xstate;                     // Exit status to be returned to
parent's wait
    int pid;                         // Process ID

    // wait_lock must be held when using this:
    struct proc *parent;            // Parent process

    // these are private to the process, so p->lock need not be held.
    uint64 kstack;                  // Virtual address of kernel stack
    uint64 sz;                      // Size of process memory (bytes)
    pagetable_t pagetable;          // User page table
    struct trapframe *trapframe;    // data page for trampoline.S
    struct context context;          // swtch() here to run process
    struct file *ofile[NFILE];      // Open files
    struct inode *cwd;               // Current directory
    char name[16];                  // Process name (debugging)
    int syscall_count;              // <--- Total number of system calls
that the current process has made
};
```

and the `syscall_count` counter had to be initialized to zero when the process is first allocated

```
// kernel/proc.c:109-127

static struct proc*
allocproc(void)
{
    struct proc *p;

    for(p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if(p->state == UNUSED) {
            goto found;
        } else {
            release(&p->lock);
        }
    }
```

```

}
return 0;

found:
p->pid = allocpid();
p->state = USED;
p->syscall_count = 0; // <---
...

```

4. To track the total syscalls of a single process, we again added a counter to syscall.c

```

// kernel/syscall.c:144-162

void
syscall(void)
{
    int num;
    struct proc *p = myproc();

    num = p->trapframe->a7;
    if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
        // Use num to lookup the system call function for num, call
it,
        // and store its return value in p->trapframe->a0
        p->trapframe->a0 = syscalls[num]();
        total_syscalls++;
        p->syscall_count++; // <---
    } else {
        printf("%d %s: unknown sys call %d\n",
            p->pid, p->name, num);
        p->trapframe->a0 = -1;
    }
}

```

5. The function procinfo is defined in `sysproc.c`, which handles input, and uniquely handles output of the struct to user space using the `copyout()` function.

```

// kernel/sysproc.c:115-128

uint64 sys_procinfo(void) {
    uint64 pinfo_addr; // Address of input pinfo struct
    struct pinfo info; // Struct to write into user space

    argaddr(0, &pinfo_addr); // Take input value from reg0 and store
into pinfo_addr

    // Write process information to info
    if (generate_procinfo(&info) < 0) return -1;
}

```

```

        // Copy info to user space
        if (copyout(myproc()->pagetable, pinfo_addr, (char *)&info,
sizeof(info)) < 0) return -1;

        return 0;
    }

```

6. The function is finally defined in `proc.c`, and assigns the values to the struct `pinfo` from various sources, and counts the pages of memory used by dividing the size of the process by page size rounded up.

```

// kernel/proc.c:735-749

int generate_procinfo(struct pinfo *p) {
    struct proc *currProc = myproc();

    if (!p || !currProc) {
        return -1;
    }

    if (currProc->parent) p->ppid = currProc->parent->pid;
    else p->ppid = 0;

    p->syscall_count = currProc->syscall_count;
    p->page_usage = PGROUNDUP(currProc->sz) / PGSIZE;

    return 0;
}

```

Lab 1 Test Output

```

xv6 kernel is booting

init: starting sh

$ lab1test 65536 2
[sysinfo] active proc: 3, syscalls: 50, free pages: 32532
[procinfo 4] ppid: 3, syscalls: 10, page usage: 21
[procinfo 5] ppid: 3, syscalls: 10, page usage: 21
[sysinfo] active proc: 5, syscalls: 242, free pages: 32478

$ lab1test 65000 10
[sysinfo] active proc: 3, syscalls: 331, free pages: 32532
[procinfo 7] ppid: 6, syscalls: 10, page usage: 20
[procinfo 8] ppid: 6, syscalls: 10, page usage: 20
[procinfo 9] ppid: 6, syscalls: 10, page usage: 20
[procinfo 10] ppid: 6, syscalls: 10, page usage: 20
[procinfo 11] ppid: 6, syscalls: 10, page usage: 20
[procinfo 12] ppid: 6, syscalls: 10, page usage: 20

```

```
[procinfo 13] ppid: 6, syscalls: 10, page usage: 20  
[procinfo 14] ppid: 6, syscalls: 10, page usage: 20  
[procinfo 15] ppid: 6, syscalls: 10, page usage: 20  
[procinfo 16] ppid: 6, syscalls: 10, page usage: 20  
[sysinfo] active proc: 13, syscalls: 1051, free pages: 32272
```

Contributions

Both of us completed the main code seperately. The report part 0 and 1 were written by Zachary, and part 2 was written by Calianna. The demo video was recorded by Zachary.